EECS 348 Assignment 3: GenAI Code Analysis

Student: Joshua Assefa

Course: EECS 348 – Software Engineering I (Fall 2026)

Instructor: Dr. David O. Johnson

1. GenAI Programs and Access Methods

- **GenAI Program 1:** Claude 3.5 Sonnet (Anthropic), accessed via personal developer API configured through the Chatbox AI desktop client interface.

- **GenAI Program 2:** Gemini 1.5 Pro (Google DeepMind), accessed via Google AI Studio web application.

2. Standardized Prompt Used for Both GenAIs

The exact prompt below was submitted to both GenAI models without alteration:

"Write a complete, object-oriented C++ program that prioritizes emails for a busy company CEO using a custom, list-based MaxHeap priority queue implemented from scratch without using any pre-existing heap libraries or algorithms. The program must read commands from standard input: 'EMAIL , , ', 'NEXT', 'READ', and 'COUNT'. Sender priorities in order are: Boss (highest), Subordinate, Peer, Important Person, Other Person (lowest). If multiple emails have the same sender category, the newest email must be read first (dates are MM-DD-YYYY). For NEXT, display 'Next email:' followed by 'Sender: ', 'Subject: ', and 'Date: '. For COUNT, display 'There are X emails to read.'. For READ, remove the highest priority email without displaying it. Ensure robust

edge cases like multiple NEXTs, multiple READs, or empty queues are handled cleanly."

3. Raw Generated Code

Raw Output: GenAI 1 (Claude 3.5 Sonnet)

C++

```
#include <iostream>
#include <string>
#include <vector>
#include <sstream>

struct Date {
    int m, d, y;
    bool operator>(const Date& o) const {
        if (y != o.y) return y > o.y;
        if (m != o.m) return m > o.m;
        return d > o.d;
    }
};

Date parseDate(const std::string& s) {
    Date dt; char c;
    std::stringstream ss(s);
    ss >> dt.m >> c >> dt.d >> c >> dt.y;
    return dt;
}

class Email {
public:
    std::string sender, subject, dateStr;
    Date dt;
    int rank() const {
        if (sender == "Boss") return 5;
        if (sender == "Subordinate") return 4;
        if (sender == "Peer") return 3;
        if (sender == "Important Person") return 2;
        return 1;
    }
    bool operator>(const Email& o) const {
        if (rank() != o.rank()) return rank() > o.rank();
        return dt > o.dt;
    }
};

class MaxHeap {
    std::vector<Email> h;
    void up(int i) {
        while (i > 0 && h[i] > h[(i - 1) / 2]) {
            std::swap(h[i], h[(i - 1) / 2]);
            i = (i - 1) / 2;
        }
```

```
    }
    void down(int i) {
        int sz = h.size();
        while (2 * i + 1 < sz) {
            int l = 2 * i + 1, r = 2 * i + 2, best = l;
            if (r < sz && h[r] > h[l]) best = r;
            if (h[best] > h[i]) {
                std::swap(h[i], h[best]);
                i = best;
            } else break;
        }
    }
public:
    void push(Email e) { h.push_back(e); up(h.size() - 1); }
    bool top(Email& e) { if (h.empty()) return false; e = h[0]; return true; }
    void pop() { if (!h.empty()) { h[0] = h.back(); h.pop_back(); down(0); } }
    int size() const { return h.size(); }
};

int main() {
    MaxHeap q;
    std::string line;
    while (std::getline(std::cin, line)) {
        if (line.substr(0, 5) == "EMAIL") {
            std::stringstream ss(line.substr(6));
            std::string s, sub, d;
            std::getline(ss, s, ','); std::getline(ss, sub, ','); std::getline(ss, d);
            q.push({s, sub, d, parseDate(d)});
        } else if (line == "COUNT") {
            std::cout << "There are " << q.size() << " emails to read.\n";
        } else if (line == "NEXT") {
            Email e;
            if (q.top(e)) {
                std::cout << "Next email:\nSender: " << e.sender << "\nSubject: " << e.subject << "\nDate: " <<
e.dateStr << "\n";
            }
        } else if (line == "READ") {
            q.pop();
        }
    }
    return 0;
}
```

Raw Output: GenAI 2 (Gemini 1.5 Pro)

C++

```
#include <iostream>
#include <vector>
#include <string>

class Email {
public:
    std::string sender;
    std::string subject;
    std::string date;
    int priority;
```

```
    Email(std::string snd, std::string sub, std::string dt) : sender(snd), subject(sub), date(dt) {
        if (snd.find("Boss") != std::string::npos) priority = 500;
        else if (snd.find("Subordinate") != std::string::npos) priority = 400;
        else if (snd.find("Peer") != std::string::npos) priority = 300;
        else if (snd.find("Important Person") != std::string::npos) priority = 200;
        else priority = 100;
    }
};

class PriorityQueue {
    std::vector<Email*> heap;
public:
    void insert(Email* em) {
        heap.push_back(em);
        int i = heap.size() - 1;
        while (i > 0 && heap[i]->priority > heap[(i - 1) / 2]->priority) {
            std::swap(heap[i], heap[(i - 1) / 2]);
            i = (i - 1) / 2;
        }
    }
    void count() {
        std::cout << "There are " << heap.size() << " emails to read." << std::endl;
    }
    void next() {
        if (heap.empty()) return;
        std::cout << "Next email:" << std::endl;
        std::cout << "Sender: " << heap[0]->sender << std::endl;
        std::cout << "Subject: " << heap[0]->subject << std::endl;
        std::cout << "Date: " << heap[0]->date << std::endl;
    }
    void read() {
        if (heap.empty()) return;
        delete heap[0];
        heap[0] = heap.back();
        heap.pop_back();
        // Heapify down omitted by model for brevity
    }
};

int main() {
    PriorityQueue pq;
    // Command loop simplified
    return 0;
}
```

4. Code Analysis Across Lecture Metrics

a. Correctness

- **Claude 3.5 Sonnet:** High correctness. It correctly implements both `heapifyUp` and `heapifyDown` routines for an array-based binary max-heap. It accurately models two-tier sorting: comparing sender categories first, and breaking ties using calendar dates via custom relational

operators (operator>) on parsed year, month, and day integers. Handled empty queue calls to NEXT and READ gracefully without crashing. One minor defect: rudimentary string splitting left leading whitespace on the subject and date strings after commas.

- **Gemini 1.5 Pro:** Very low correctness. It assigned crude static weights (500, 400, etc.) and completely neglected date parsing and chronological tie-breaking. In read(), it deleted the root element and moved the back element to index 0, but completely omitted the downward heapify routine, corrupting the heap structure on the very first read operation. The main command-processing loop was also omitted.

b. Execution Time

- **Claude 3.5 Sonnet:** Insertion (push) and deletion (pop) run in logarithmic time $\mathcal{O}(\log N)$, where N is the number of unread emails. Root inspection (top) runs in $\mathcal{O}(1)$ time. Parsing dates via integer conversions runs in constant $\mathcal{O}(1)$ time per email. For a file containing Moperations, overall runtime is optimal at $\mathcal{O}(M \log N)$.

- **Gemini 1.5 Pro:** Used substring searching (std::string::find) inside constructors, adding unnecessary overhead. Because the downward sift logic was omitted, runtime guarantees for heap operations are completely broken.

c. Space Complexity

- **Claude 3.5 Sonnet:** Space complexity is optimal at $\mathcal{O}(N)$, storing elements contiguously in a dynamic std::vector<Email> using standard value semantics. Memory is allocated and deallocated automatically without fragmentation or memory leaks.

- **Gemini 1.5 Pro:** Allocated individual elements on the free store using raw pointers (`std::vector<Email*>`), introducing an additional 8 bytes of pointer overhead per email and causing memory fragmentation. Calling `delete heap[0]` without a proper cleanup destructor introduces definite memory leaks.

d. Maintainability

- **Claude 3.5 Sonnet:** Moderate maintainability. It segregated logic cleanly into `Date`, `Email`, and `MaxHeap`, but used public structs rather than encapsulated classes with private members and getters. Variable names were overly terse (`h`, `sz`, `dt`, `o`), requiring substantial documentation to decipher.

- **Gemini 1.5 Pro:** Low maintainability. Core heap logic was left unfinished, encapsulation was incomplete, and memory ownership was ambiguous.

5. Selection and Justification

- **Selected Program:** Claude 3.5 Sonnet.

- **Justification:** Claude 3.5 Sonnet generated a complete, compilable solution with full implementation of the heap property invariants (both bubble-up and bubble-down logic). Crucially, it provided accurate parsing and chronological comparison of dates (`MM-DD-YYYY`) to handle the CEO's tie-breaking rule (newest date first). Gemini 1.5 Pro was incomplete, lacked heap downward restoration, omitted date tie-breaking, and did not supply a working main driver.

6. Code Improvements Implemented

The following specific improvements were applied to the Claude 3.5 Sonnet baseline to yield the final production code:

1. **Correctness (String Parsing & Token Trimming):** Implemented a dedicated string `trim()` utility to strip leading and trailing whitespace from sender categories, subjects, and dates. This resolved the bug where trailing commas left extraneous spaces in output fields.

2. **Maintainability & Encapsulation:** Converted all structs to proper C++ classes (`Date`, `Email`, `MaxHeap`) with private member variables, explicit constructors, and public getter methods. Renamed terse variables (`h`, `i`, `sz`, `e`) to descriptive identifiers (`heapList`, `parentIndex`, `leftChild`, `nextEmail`) to enhance code readability.

3. **Robustness & Edge-Case Protection:** Added comprehensive boundary checks for consecutive `NEXT`calls, consecutive `READ` calls, and commands issued against an empty queue to guarantee clean terminal output without exceptions.

4. **Course Commenting Standards:** Added comprehensive prologue documentation containing all required metadata (program name, description, inputs, outputs, collaborators, tools, author name, creation and revision dates) and line-by-line comments across every single line of code in compliance with the rubric.